```
<#
    Redact-FileContent.ps1

    Unified replacement/redaction engine for files, recursively through subfolders.
    All modes below share the same file-processing, dry-run and backup logic.

      1. Literal/regex replacements ($Replacements below) - always applied.
         Each key is a regex pattern, each value is what to replace it with.
         (Companion to Search-FilesForWord.ps1 - use that one first to preview matches.)

      2. -AnonymizeSwitchNames - finds whitespace-delimited tokens that start with
         "S"/"s" and end with "nn" (letters, digits, and special characters allowed
         in between, e.g. S-DC1-01nn) and replaces each with a sequential generic
         placeholder (SWITCH001, SWITCH002, ...).

      3. -AnonymizeUsernames - finds tokens like "user22351" (literal "user" followed
         directly by digits) and replaces each with a sequential placeholder
         (USER001, USER002, ...).

      4. -AnonymizeIPs - finds IPv4 addresses and replaces each with a sequential
         placeholder (IP001, IP002, ...).

      5. -StripPortDescriptions - removes the description text from interface lines
         such as "GigabitEthernet1/0/1 - TO_SWITCH001", leaving just the port name
         ("GigabitEthernet1/0/1"). Covers GigabitEthernet, TenGigabitEthernet,
         TwentyFiveGigabitEthernet, FastEthernet and Ethernet ports.

    For modes 2-4, the same original value always maps to the same placeholder
    within a single run (each category has its own counter), but nothing is
    persisted between runs - running the script again assigns fresh numbers.

    All write modes support -DryRun (report what would change, touch nothing) and
    -Backup (copy each modified file's original content aside before overwriting it).

    USAGE EXAMPLES:

        # Preview the literal replacements only, no files modified
        .\Redact-FileContent.ps1 -Path "D:\" -DryRun

        # Apply literal replacements for real, keeping backups
        .\Redact-FileContent.ps1 -Path "D:\" -Backup

        # Anonymize switch names, usernames and IPs, and strip port descriptions
        .\Redact-FileContent.ps1 -Path "D:\" -AnonymizeSwitchNames -AnonymizeUsernames -AnonymizeIPs -StripPortD
    escriptions -Backup

        # Anonymization only - skip the literal $Replacements table
        .\Redact-FileContent.ps1 -Path "D:\" -AnonymizeIPs -Replacements @{} -DryRun

        # Limit to specific file types
        .\Redact-FileContent.ps1 -Path "D:\Logs" -Include *.log,*.txt -AnonymizeUsernames -DryRun
#>

[CmdletBinding(SupportsShouldProcess = $true)]
param(
    # Root folder to process. Defaults to the current directory.
    [string]$Path = ".",

    # Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
    # Leave empty to process every file.
    [string[]]$Include = @("*"),

    # Ordered map of regex pattern -> replacement text, applied on every line.
    [System.Collections.Specialized.OrderedDictionary]$Replacements = [ordered]@{
        "ass" = "aaa"
    },
```

```
    # Anonymize switch/device name tokens matching -SwitchNamePattern.
    [switch]$AnonymizeSwitchNames,
    [string]$SwitchNamePattern = '(?<!\S)[Ss][^\s]*nn(?!\S)',
    [string]$SwitchNamePrefix = "SWITCH",

    # Anonymize username tokens matching -UsernamePattern.
    [switch]$AnonymizeUsernames,
    [string]$UsernamePattern = '(?<!\S)user\d+(?!\S)',
    [string]$UsernamePrefix = "USER",

    # Anonymize IPv4 addresses.
    [switch]$AnonymizeIPs,
    [string]$IPPattern = '(?<!\d)(?:(?:25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(?:25[0-5]|2[0-4]\d|1?\d?\d)(?!\d)',
    [string]$IPPrefix = "IP",

    # Strip the description text off interface lines, e.g.
    # "GigabitEthernet1/0/1 - TO_SWITCH001" -> "GigabitEthernet1/0/1".
    [switch]$StripPortDescriptions,
    [string]$PortDescriptionPattern = '^(?<port>(?:TwentyFiveGigabitEthernet|TenGigabitEthernet|GigabitEthernet|
    FastEthernet|Ethernet)\d+(?:/\d+)*)\s*-\s*.*$',

    # Report matches and what would change, without modifying any file.
    [switch]$DryRun,

    # Copy each file's original content into -BackupFolder before overwriting it.
    [switch]$Backup,

    # Folder to hold backups when -Backup is used. Defaults to a timestamped
    # folder created next to -Path. Relative structure under -Path is preserved.
    [string]$BackupFolder
)

if (-not (Test-Path -LiteralPath $Path)) {
    Write-Error "Path not found: $Path"
    return
}

if ($Backup -and -not $BackupFolder) {
    $BackupFolder = Join-Path (Resolve-Path -LiteralPath $Path) "_backup_$(Get-Date -Format 'yyyyMMdd_HHmmss')"
}

$resolvedRoot = (Resolve-Path -LiteralPath $Path).ProviderPath
$allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction SilentlyContinue

$ignoreCase = [System.Text.RegularExpressions.RegexOptions]::IgnoreCase
$switchNameRegex = [regex]::new($SwitchNamePattern, $ignoreCase)
$usernameRegex = [regex]::new($UsernamePattern, $ignoreCase)
$ipRegex = [regex]::new($IPPattern)
$portDescriptionRegex = [regex]::new($PortDescriptionPattern, $ignoreCase)

# Original value (as first encountered) -> generic placeholder, one map per category.
# Consistent for this run only - not persisted between runs.
$switchNameMap = [ordered]@{}
$usernameMap = [ordered]@{}
$ipMap = [ordered]@{}
$switchNameIndex = 1
$usernameIndex = 1
$ipIndex = 1

function Get-PlaceholderName {
    param(
        [string]$OriginalValue,
        [System.Collections.Specialized.OrderedDictionary]$Map,
        [string]$Prefix,
        [string]$IndexVarName
    )
```

```
    if (-not $Map.Contains($OriginalValue)) {
        $index = Get-Variable -Name $IndexVarName -Scope Script -ValueOnly
        $Map[$OriginalValue] = "{0}{1:D3}" -f $Prefix, $index
        Set-Variable -Name $IndexVarName -Scope Script -Value ($index + 1)
    }
    return $Map[$OriginalValue]
}

$changedCount = 0
$scannedCount = 0

foreach ($fileItem in $allFiles) {
    $scannedCount++

    $content = Get-Content -LiteralPath $fileItem.FullName -ErrorAction SilentlyContinue
    if ($null -eq $content) {
        continue
    }

    $fileMatchCount = 0
    $updatedContent = foreach ($row in $content) {
        $currentRow = $row

        foreach ($search in $Replacements.Keys) {
            $lineMatches = [regex]::Matches($currentRow, $search).Count
            if ($lineMatches -gt 0) {
                $fileMatchCount += $lineMatches
                $currentRow = $currentRow -replace $search, $Replacements[$search]
            }
        }

        if ($StripPortDescriptions -and $portDescriptionRegex.IsMatch($currentRow)) {
            $fileMatchCount++
            $currentRow = $portDescriptionRegex.Replace($currentRow, '${port}')
        }

        if ($AnonymizeSwitchNames) {
            $currentRow = $switchNameRegex.Replace($currentRow, {
                param($match)
                $script:fileMatchCount++
                Get-PlaceholderName -OriginalValue $match.Value -Map $switchNameMap -Prefix $SwitchNamePrefix -I
    ndexVarName "switchNameIndex"
            })
        }

        if ($AnonymizeUsernames) {
            $currentRow = $usernameRegex.Replace($currentRow, {
                param($match)
                $script:fileMatchCount++
                Get-PlaceholderName -OriginalValue $match.Value -Map $usernameMap -Prefix $UsernamePrefix -Index
    VarName "usernameIndex"
            })
        }

        if ($AnonymizeIPs) {
            $currentRow = $ipRegex.Replace($currentRow, {
                param($match)
                $script:fileMatchCount++
                Get-PlaceholderName -OriginalValue $match.Value -Map $ipMap -Prefix $IPPrefix -IndexVarName "ipI
    ndex"
            })
        }

        $currentRow
    }

    if ($fileMatchCount -eq 0) {
```

```
        continue
    }

    $changedCount++

    if ($DryRun) {
        Write-Host "[DryRun] Would update: $($fileItem.FullName) ($fileMatchCount match(es))" -ForegroundColor D
    arkYellow
        continue
    }

    if (-not $PSCmdlet.ShouldProcess($fileItem.FullName, "Replace $fileMatchCount match(es)")) {
        continue
    }

    if ($Backup) {
        $relativePath = $fileItem.FullName.Substring($resolvedRoot.Length).TrimStart('\', '/')
        $backupPath = Join-Path $BackupFolder $relativePath
        $backupDir = Split-Path -Path $backupPath -Parent
        if (-not (Test-Path -LiteralPath $backupDir)) {
            New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
        }
        Copy-Item -LiteralPath $fileItem.FullName -Destination $backupPath -Force
    }

    Write-Host "Working on: $($fileItem.FullName) ($fileMatchCount match(es))" -ForegroundColor DarkYellow
    $updatedContent | Set-Content -LiteralPath $fileItem.FullName
}

Write-Host ""
if ($DryRun) {
    Write-Host "Dry run complete. $changedCount of $scannedCount file(s) would be updated." -ForegroundColor Gre
    en
} else {
    Write-Host "Complete successfully! $changedCount of $scannedCount file(s) updated." -ForegroundColor Green
    if ($Backup) {
        Write-Host "Backups saved under: $BackupFolder" -ForegroundColor Green
    }
}

function Write-NameMapping {
    param([string]$Title, [System.Collections.Specialized.OrderedDictionary]$Map)

    if ($Map.Count -eq 0) {
        return
    }
    Write-Host "`n$Title mapping for this run:" -ForegroundColor Cyan
    $Map.GetEnumerator() | ForEach-Object {
        Write-Host ("  {0}  ->  {1}" -f $_.Key, $_.Value)
    }
}

if ($AnonymizeSwitchNames) { Write-NameMapping -Title "Switch name" -Map $switchNameMap }
if ($AnonymizeUsernames) { Write-NameMapping -Title "Username" -Map $usernameMap }
if ($AnonymizeIPs) { Write-NameMapping -Title "IP address" -Map $ipMap }
```